Model selection 🧬

The objective of this notebook was to build a regression model to predict the price of diamonds based on their physical and visual characteristics. To achieve this goal, we used several regression models and compared their performance. The dataset was loaded, preprocessed, and then we applied different regression models to predict the price. To ensure the best model is selected, we applied k-folds and observed the error metrics and R2 values.

Importing Libraries 🛠️

We will start by importing the necessary libraries that we will be using throughout the notebook.

- `pandas` - for data manipulation and analysis
- `numpy` - for numerical operations
- `time` - for time-related operations
- `matplotlib` - for data visualization
- `seaborn` - for advanced data visualization
- `train_test_split` - for splitting the dataset into training and testing sets
- `KFold` - for cross-validation
- `GradientBoostingRegressor` - for Gradient Boosting
- `RandomForestRegressor` - for Random Forest
- `ExtraTreesRegressor` - for Extra Trees
- `AdaBoostRegressor` - for AdaBoost
- `DecisionTreeRegressor` - for Decision Tree
- `LassoLars` - for Lasso Regression
- `Ridge` - for Ridge Regression
- `BayesianRidge` - for Bayesian Ridge Regression
- `Lasso` - for Lasso Regression
- `LinearRegression` - for Linear Regression
- `HuberRegressor` - for Huber Regression
- `PassiveAggressiveRegressor` - for Passive Aggressive Regression
- `ElasticNet` - for Elastic Net Regression
- `KNeighborsRegressor` - for K-Nearest Neighbors Regression
- `DummyRegressor` - for Dummy Regression
- `LGBMRegressor` - for LightGBM Regression
- `XGBRegressor` - for XGBoost Regression
- `CatBoostRegressor` - for CatBoost Regression
- `mean_absolute_error` - for Mean Absolute Error
- `mean_squared_error` - for Mean Squared Error
- `r2_score` - for R-Squared Score

In [ ]:
```
import pandas as pd
import numpy as np
import time
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, KFold
from sklearn.ensemble import (
    GradientBoostingRegressor,
    RandomForestRegressor,
    ExtraTreesRegressor,
    AdaBoostRegressor,
)
from sklearn.tree import DecisionTreeRegressor
from sklearn.linear_model import (
    LassoLars,
    Ridge,
    BayesianRidge,
    Lasso,
    LinearRegression,
    HuberRegressor,
    PassiveAggressiveRegressor,
    ElasticNet,
)
from sklearn.neighbors import KNeighborsRegressor
from sklearn.dummy import DummyRegressor
from lightgbm import LGBMRegressor
from xgboost import XGBRegressor
from catboost import CatBoostRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

Loading Data 📁

Next, we will load the dataset using pandas read_parquet function.

In [ ]:
```
df = pd.read_csv("./../data/transform_diamonds.csv")
df
```

Out[ ]:

	Shape_CUSHION	Shape_EMERALD	Shape_HEART	Shape_MARQUISE	Shape_OVAL	Shape_PEAR	Shape_PRINCESS	Shape_ROUND	Clarity	Colour	Cut	Polish	Symmetry	Fluorescence	Weight	Length	Width	Depth	Price
0	1	0	0	0	0	0	0	0	3	17	3	3	2	0	0.549805	5.050781	4.351562	2.939453	1378.65
1	1	0	0	0	0	0	0	0	8	19	3	3	2	2	0.500000	4.601562	4.308594	2.919922	1379.74
2	1	0	0	0	0	0	0	0	5	14	3	3	2	0	0.509766	4.710938	4.351562	2.939453	1380.19
3	1	0	0	0	0	0	0	0	5	14	3	3	2	0	0.500000	4.910156	4.261719	2.880859	1380.61
4	1	0	0	0	0	0	0	0	4	18	3	2	2	0	0.529785	4.699219	4.460938	3.009766	1383.13
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
5890	0	0	0	0	0	0	0	0	1	4	17	3	3	3	0.500000	5.000000	5.039062	3.179688	2453.10
5891	0	0	0	0	0	0	0	0	1	4	17	3	3	3	0.500000	5.031250	5.058594	3.160156	2453.10
5892	0	0	0	0	0	0	0	0	1	5	18	2	3	2	0.500000	4.929688	4.960938	3.189453	2453.20
5893	0	0	0	0	0	0	0	0	1	4	14	3	3	3	0.520020	5.109375	5.128906	3.199219	2453.41
5894	0	0	0	0	0	0	0	0	1	6	14	3	3	2	0.500000	5.089844	5.128906	3.119141	2453.69

5895 rows × 19 columns

Data Exploration 🔎

Let's explore the data by checking the columns and their data types.

In [ ]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5895 entries, 0 to 5894
Data columns (total 19 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Shape_CUSHION   5895 non-null   int64
 1   Shape_EMERALD   5895 non-null   int64
 2   Shape_HEART     5895 non-null   int64
 3   Shape_MARQUISE  5895 non-null   int64
 4   Shape_OVAL      5895 non-null   int64
 5   Shape_PEAR      5895 non-null   int64
 6   Shape_PRINCESS  5895 non-null   int64
 7   Shape_ROUND     5895 non-null   int64
 8   Clarity         5895 non-null   int64
 9   Colour          5895 non-null   int64
 10  Cut             5895 non-null   int64
 11  Polish          5895 non-null   int64
 12  Symmetry        5895 non-null   int64
 13  Fluorescence    5895 non-null   int64
 14  Weight          5895 non-null   float64
 15  Length          5895 non-null   float64
 16  Width           5895 non-null   float64
 17  Depth           5895 non-null   float64
 18  Price           5895 non-null   float64
dtypes: float64(5), int64(14)
memory usage: 875.2 KB
```

Define list of models 🤖

In this project we are using this regression models

- Gradient Boosting Regressor
- Random Forest Regressor
- CatBoost Regressor
- Light Gradient Boosting Machine
- Extra Trees Regressor
- AdaBoost Regressor
- Extreme Gradient Boosting
- Lasso Least Angle Regression
- Ridge Regression
- Bayesian Ridge
- Least Angle Regression
- Lasso Regression
- Linear Regression
- Huber Regressor
- Decision Tree Regressor
- Orthogonal Matching Pursuit
- Passive Aggressive Regressor
- Elastic Net
- K Neighbors Regressor
- Dummy Regressor

In [ ]:
```python
models = {
    "Gradient Boosting Regressor": GradientBoostingRegressor(),
    "Random Forest Regressor": RandomForestRegressor(),
    "CatBoost Regressor": CatBoostRegressor(verbose=0, allow_writing_files=False),
    "Light Gradient Boosting Machine": LGBMRegressor(),
    "Extra Trees Regressor": ExtraTreesRegressor(),
    "AdaBoost Regressor": AdaBoostRegressor(),
    "Extreme Gradient Boosting": XGBRegressor(),
    "Lasso Least Angle Regression": LassoLars(),
    "Ridge Regression": Ridge(),
    "Bayesian Ridge": BayesianRidge(),
    "Least Angle Regression": LassoLars(),
    "Lasso Regression": Lasso(),
    "Linear Regression": LinearRegression(),
    "Huber Regressor": HuberRegressor(max_iter=1000),
    "Decision Tree Regressor": DecisionTreeRegressor(),
    "Orthogonal Matching Pursuit": PassiveAggressiveRegressor(),
    "Passive Aggressive Regressor": PassiveAggressiveRegressor(),
    "Elastic Net": ElasticNet(),
    "K Neighbors Regressor": KNeighborsRegressor(),
    "Dummy Regressor": DummyRegressor(),
}
```

Define the k-fold cross-validator ✊

We will be using the `KFold` function from the `sklearn.model_selection` library to split the data into `k` consecutive folds.

In [ ]:
```python
k_fold = KFold(n_splits=3, shuffle=True, random_state=42)
```

Split the dataset into training and testing sets 📊

We will be using the `train_test_split` function from the `sklearn.model_selection` library to split the dataset into a training set and a testing set.

In [ ]:
```python
X = df.drop("Price", axis=1)
y = df["Price"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Cross-Validation and Model Evaluation Metrics 🔮 📊

This code block contains a loop that performs k-fold cross-validation to evaluate the performance of multiple regression models. The loop iterates over a dictionary of models, each of which is trained on k-1 folds of the training set and evaluated on the remaining fold. The loop records the mean absolute error (MAE) 👌, mean squared error (MSE) 🔍, root mean squared error (RMSE) 🔢, R-squared (R2) 📈, and the time taken (TT) ⏰ to fit and evaluate each model. The loop then evaluates each model on the test set and records the evaluation metrics. Finally, the evaluation metrics for each model on the test set are appended to lists for later analysis.

In [ ]:
```python
mae = []
mse = []
rmse = []
r2 = []
tt = []

test_mae = []
test_mse = []
test_rmse = []
test_r2 = []
test_tt = []
```

```python
for model_name, model in models.items():
    mae_scores = []
    mse_scores = []
    rmse_scores = []
    r2_scores = []
    tt_scores = []
    print(f"Model name: {model_name}\n")
    print("Train model")

    for k, (train_index, val_index) in enumerate(k_fold.split(X_train, y_train)):
        X_train_fold, X_val_fold = X_train.iloc[train_index], X_train.iloc[val_index]
        y_train_fold, y_val_fold = y_train.iloc[train_index], y_train.iloc[val_index]

        start_time = time.time()
        model.fit(X_train_fold, y_train_fold)
        y_pred = model.predict(X_val_fold)
        end_time = time.time()

        mae_scores.append(round(mean_absolute_error(y_val_fold, y_pred), 4))
        mse_scores.append(round(mean_squared_error(y_val_fold, y_pred), 4))
        rmse_scores.append(
            round(mean_squared_error(y_val_fold, y_pred, squared=False), 4)
        )
        r2_scores.append(round(r2_score(y_val_fold, y_pred), 4))
        tt_scores.append(round(end_time - start_time, 4))

        print(
            f"Fold: {k}\tMAE: {mae_scores[-1]:.4f}\tMSE: {mse_scores[-1]:.4f}\tRMSE: {rmse_scores[-1]:.4f}\tR2: {r2_scores[-1]:.4f}\tTT: {tt_scores[-1]:.4f}"
        )

    mae.append(np.mean(mae_scores))
    mse.append(np.mean(mse_scores))
    rmse.append(np.mean(rmse_scores))
    r2.append(np.mean(r2_scores))
    tt.append(np.mean(tt_scores))

    print("\nTest model")
    start_time = time.time()
    y_pred = model.predict(X_test)
    end_time = time.time()

    mae_scores.append(round(mean_absolute_error(y_test, y_pred), 4))
    mse_scores.append(round(mean_squared_error(y_test, y_pred), 4))
    rmse_scores.append(round(mean_squared_error(y_test, y_pred, squared=False), 4))
    r2_scores.append(round(r2_score(y_test, y_pred), 4))
    tt_scores.append(round(end_time - start_time, 4))

    print(
        f"Fold: -\tMAE: {mae_scores[-1]:.4f}\tMSE: {mse_scores[-1]:.4f}\tRMSE: {rmse_scores[-1]:.4f}\tR2: {r2_scores[-1]:.4f}\tTT: {tt_scores[-1]:.4f}"
    )

    test_mae.append(mae_scores[-1])
    test_mse.append(mse_scores[-1])
    test_rmse.append(rmse_scores[-1])
    test_r2.append(r2_scores[-1])
    test_tt.append(tt_scores[-1])

    print("\n" + "=" * 100 + "\n")
```

```
Model name: Gradient Boosting Regressor

Train model
Fold: 0 MAE: 224.7372   MSE: 261971.1195        RMSE: 511.8311  R2: 0.8873      TT: 0.4476
Fold: 1 MAE: 253.1958   MSE: 454447.9312        RMSE: 674.1275  R2: 0.8531      TT: 0.2913
Fold: 2 MAE: 230.6118   MSE: 351107.1568        RMSE: 592.5430  R2: 0.8685      TT: 0.2761

Test model
Fold: - MAE: 246.8139   MSE: 426968.2167        RMSE: 653.4281  R2: 0.8469      TT: 0.0023

========================================================================================================

Model name: Random Forest Regressor

Train model
Fold: 0 MAE: 201.5742   MSE: 295963.5875        RMSE: 544.0254  R2: 0.8726      TT: 1.0430
Fold: 1 MAE: 225.8366   MSE: 486818.0258        RMSE: 697.7235  R2: 0.8426      TT: 1.0460
Fold: 2 MAE: 217.5591   MSE: 388202.5748        RMSE: 623.0590  R2: 0.8546      TT: 1.1533

Test model
Fold: - MAE: 226.8002   MSE: 389923.9129        RMSE: 624.4389  R2: 0.8602      TT: 0.0294

========================================================================================================

Model name: CatBoost Regressor

Train model
Fold: 0 MAE: 177.3680   MSE: 207877.0197        RMSE: 455.9353  R2: 0.9105      TT: 2.1812
Fold: 1 MAE: 197.9240   MSE: 340714.3270        RMSE: 583.7074  R2: 0.8898      TT: 1.1907
Fold: 2 MAE: 195.7100   MSE: 310586.9766        RMSE: 557.3033  R2: 0.8837      TT: 1.1858

Test model
Fold: - MAE: 206.8450   MSE: 351926.3357        RMSE: 593.2338  R2: 0.8738      TT: 0.0038

========================================================================================================

Model name: Light Gradient Boosting Machine

Train model
Fold: 0 MAE: 204.1293   MSE: 348110.1059        RMSE: 590.0086  R2: 0.8502      TT: 1.2352
Fold: 1 MAE: 221.4395   MSE: 439893.5187        RMSE: 663.2447  R2: 0.8578      TT: 0.3929
Fold: 2 MAE: 214.2691   MSE: 388797.4080        RMSE: 623.5362  R2: 0.8544      TT: 0.6105

Test model
Fold: - MAE: 231.0250   MSE: 446256.2799        RMSE: 668.0242  R2: 0.8400      TT: 0.0039

========================================================================================================

Model name: Extra Trees Regressor

Train model
Fold: 0 MAE: 193.7815   MSE: 303886.1937        RMSE: 551.2587  R2: 0.8692      TT: 0.8804
Fold: 1 MAE: 215.7960   MSE: 459757.0182        RMSE: 678.0538  R2: 0.8514      TT: 0.7504
Fold: 2 MAE: 205.4680   MSE: 337184.8951        RMSE: 580.6762  R2: 0.8737      TT: 0.7243

Test model
Fold: - MAE: 223.1459   MSE: 429009.1899        RMSE: 654.9879  R2: 0.8462      TT: 0.0311

========================================================================================================

Model name: AdaBoost Regressor

Train model
Fold: 0 MAE: 723.7084   MSE: 960066.1555        RMSE: 979.8297  R2: 0.5868      TT: 0.2641
Fold: 1 MAE: 490.3675   MSE: 820867.0108        RMSE: 906.0171  R2: 0.7346      TT: 0.0722
Fold: 2 MAE: 678.8356   MSE: 924565.6057        RMSE: 961.5433  R2: 0.6538      TT: 0.1656

Test model
Fold: - MAE: 725.5011   MSE: 1021387.9262       RMSE: 1010.6374 R2: 0.6339      TT: 0.0132

========================================================================================================

Model name: Extreme Gradient Boosting

Train model
Fold: 0 MAE: 195.3571   MSE: 297101.5617        RMSE: 545.0702  R2: 0.8721      TT: 1.1355
Fold: 1 MAE: 226.9826   MSE: 510168.7475        RMSE: 714.2610  R2: 0.8351      TT: 0.4636
Fold: 2 MAE: 211.3979   MSE: 424978.9608        RMSE: 651.9041  R2: 0.8409      TT: 0.7426

Test model
Fold: - MAE: 219.3328   MSE: 402888.7158        RMSE: 634.7352  R2: 0.8556      TT: 0.0060

========================================================================================================

Model name: Lasso Least Angle Regression

Train model
Fold: 0 MAE: 422.3800   MSE: 708668.1096        RMSE: 841.8243  R2: 0.6950      TT: 0.2627
Fold: 1 MAE: 416.9615   MSE: 826345.5484        RMSE: 909.0355  R2: 0.7328      TT: 0.0159
Fold: 2 MAE: 404.6732   MSE: 705413.2795        RMSE: 839.8888  R2: 0.7358      TT: 0.0103

Test model
Fold: - MAE: 447.3380   MSE: 820998.1042        RMSE: 906.0895  R2: 0.7057      TT: 0.0031

========================================================================================================

Model name: Ridge Regression

Train model
Fold: 0 MAE: 423.4085   MSE: 706067.0310        RMSE: 840.2779  R2: 0.6961      TT: 0.0575
Fold: 1 MAE: 420.5534   MSE: 829266.5141        RMSE: 910.6407  R2: 0.7319      TT: 0.0089
Fold: 2 MAE: 404.9881   MSE: 702849.6572        RMSE: 838.3613  R2: 0.7368      TT: 0.0060

Test model
Fold: - MAE: 447.5752   MSE: 817691.7782        RMSE: 904.2631  R2: 0.7069      TT: 0.0019

========================================================================================================

Model name: Bayesian Ridge

Train model
Fold: 0 MAE: 425.4249   MSE: 707898.7468        RMSE: 841.3672  R2: 0.6953      TT: 0.0347
Fold: 1 MAE: 421.8614   MSE: 825978.7875        RMSE: 908.8338  R2: 0.7330      TT: 0.0096
Fold: 2 MAE: 406.3053   MSE: 701626.7528        RMSE: 837.6316  R2: 0.7373      TT: 0.0087

Test model
Fold: - MAE: 448.9835   MSE: 818388.7856        RMSE: 904.6484  R2: 0.7066      TT: 0.0018

========================================================================================================

Model name: Least Angle Regression

Train model
Fold: 0 MAE: 422.3800   MSE: 708668.1096        RMSE: 841.8243  R2: 0.6950      TT: 0.0123
```

```
Fold: 1 MAE: 416.9615   MSE: 826345.5484       RMSE: 909.0355  R2: 0.7328      TT: 0.0116
Fold: 2 MAE: 404.6732   MSE: 705413.2795       RMSE: 839.8888  R2: 0.7358      TT: 0.0109

Test model
Fold: - MAE: 447.3380   MSE: 820998.1042       RMSE: 906.0895  R2: 0.7057      TT: 0.0019

========================================================================================================

Model name: Lasso Regression

Train model
Fold: 0 MAE: 422.3735   MSE: 708676.8231       RMSE: 841.8295  R2: 0.6950      TT: 0.0553
Fold: 1 MAE: 416.9500   MSE: 826353.1067       RMSE: 909.0397  R2: 0.7328      TT: 0.0129
Fold: 2 MAE: 404.6703   MSE: 705439.0455       RMSE: 839.9042  R2: 0.7358      TT: 0.0168

Test model
Fold: - MAE: 447.3344   MSE: 821009.7778       RMSE: 906.0959  R2: 0.7057      TT: 0.0018

========================================================================================================

Model name: Linear Regression

Train model
Fold: 0 MAE: 426.6375   MSE: 708927.0479       RMSE: 841.9781  R2: 0.6949      TT: 0.0826
Fold: 1 MAE: 422.7728   MSE: 824768.0561       RMSE: 908.1674  R2: 0.7333      TT: 0.0085
Fold: 2 MAE: 407.0128   MSE: 701057.4760       RMSE: 837.2918  R2: 0.7375      TT: 0.0065

Test model
Fold: - MAE: 449.7303   MSE: 819099.0030       RMSE: 905.0409  R2: 0.7064      TT: 0.0020

========================================================================================================

Model name: Huber Regressor

Train model
Fold: 0 MAE: 359.8158   MSE: 762101.7407       RMSE: 872.9844  R2: 0.6720      TT: 0.6201
Fold: 1 MAE: 374.8378   MSE: 1022442.1747      RMSE: 1011.1588 R2: 0.6694      TT: 0.6288
Fold: 2 MAE: 363.4251   MSE: 875785.4388       RMSE: 935.8341  R2: 0.6720      TT: 0.5150

Test model
Fold: - MAE: 395.6735   MSE: 976473.1531       RMSE: 988.1666  R2: 0.6500      TT: 0.0035

========================================================================================================

Model name: Decision Tree Regressor

Train model
Fold: 0 MAE: 265.2732   MSE: 665030.3870       RMSE: 815.4940  R2: 0.7138      TT: 0.0325
Fold: 1 MAE: 300.1510   MSE: 812808.7068       RMSE: 901.5590  R2: 0.7372      TT: 0.0307
Fold: 2 MAE: 279.2653   MSE: 822068.5852       RMSE: 906.6800  R2: 0.6922      TT: 0.0316

Test model
Fold: - MAE: 274.8801   MSE: 551162.8245       RMSE: 742.4034  R2: 0.8024      TT: 0.0020

========================================================================================================

Model name: Orthogonal Matching Pursuit

Train model
Fold: 0 MAE: 377.9980   MSE: 791352.8749       RMSE: 889.5802  R2: 0.6594      TT: 0.0571
Fold: 1 MAE: 378.3900   MSE: 1022731.0912      RMSE: 1011.3017 R2: 0.6693      TT: 0.0419
Fold: 2 MAE: 415.4699   MSE: 987375.3095       RMSE: 993.6676  R2: 0.6303      TT: 0.0298

Test model
Fold: - MAE: 427.7810   MSE: 1066703.1642      RMSE: 1032.8132 R2: 0.6176      TT: 0.0017

========================================================================================================

Model name: Passive Aggressive Regressor

Train model
Fold: 0 MAE: 383.6072   MSE: 801292.7257       RMSE: 895.1496  R2: 0.6551      TT: 0.0386
Fold: 1 MAE: 562.6122   MSE: 1100282.0093      RMSE: 1048.9433 R2: 0.6443      TT: 0.0423
Fold: 2 MAE: 481.6463   MSE: 1034519.5129      RMSE: 1017.1133 R2: 0.6126      TT: 0.0401

Test model
Fold: - MAE: 489.4734   MSE: 1110460.5446      RMSE: 1053.7839 R2: 0.6019      TT: 0.0017

========================================================================================================

Model name: Elastic Net

Train model
Fold: 0 MAE: 455.9304   MSE: 1027700.4068      RMSE: 1013.7556 R2: 0.5577      TT: 0.0099
Fold: 1 MAE: 495.5147   MSE: 1491443.1775      RMSE: 1221.2466 R2: 0.5178      TT: 0.0067
Fold: 2 MAE: 457.8132   MSE: 1194068.2432      RMSE: 1092.7343 R2: 0.5529      TT: 0.0063

Test model
Fold: - MAE: 484.7143   MSE: 1284175.4302      RMSE: 1133.2146 R2: 0.5397      TT: 0.0037

========================================================================================================

Model name: K Neighbors Regressor

Train model
Fold: 0 MAE: 331.9459   MSE: 526883.9191       RMSE: 725.8677  R2: 0.7732      TT: 0.3152
Fold: 1 MAE: 359.4695   MSE: 694365.6966       RMSE: 833.2861  R2: 0.7755      TT: 0.0204
Fold: 2 MAE: 349.6151   MSE: 772101.9009       RMSE: 878.6933  R2: 0.7109      TT: 0.0453

Test model
Fold: - MAE: 360.9694   MSE: 739841.2326       RMSE: 860.1402  R2: 0.7348      TT: 0.0159

========================================================================================================

Model name: Dummy Regressor

Train model
Fold: 0 MAE: 821.5554   MSE: 2328095.8206      RMSE: 1525.8099 R2: -0.0020     TT: 0.0003
Fold: 1 MAE: 877.4982   MSE: 3095972.0884      RMSE: 1759.5375 R2: -0.0010     TT: 0.0003
Fold: 2 MAE: 855.9609   MSE: 2670586.7656      RMSE: 1634.1930 R2: -0.0001     TT: 0.0003

Test model
Fold: - MAE: 859.7680   MSE: 2790455.0826      RMSE: 1670.4655 R2: -0.0003     TT: 0.0001

========================================================================================================
```

Results table showing the performance metrics of different regression models in train dataset

In [ ]:
```
results = pd.DataFrame(
    {
        "Model": list(models.keys()),
        "MAE": mae,
```

```
        "MSE": mse,
        "RMSE": rmse,
        "R2": r2,
        "TT (Sec)": tt,
    }
)

results.sort_values("R2", axis=0, ascending=False).reset_index(drop=True)
```

Out[ ]:

	Model	MAE	MSE	RMSE	R2	TT (Sec)
0	CatBoost Regressor	190.334000	2.863928e+05	532.315333	0.894667	1.519233
1	Gradient Boosting Regressor	236.181600	3.558421e+05	592.833867	0.869633	0.338333
2	Extra Trees Regressor	205.015167	3.669427e+05	603.329567	0.864767	0.785033
3	Random Forest Regressor	214.989967	3.903281e+05	621.602633	0.856600	1.080767
4	Light Gradient Boosting Machine	213.279300	3.922670e+05	625.596500	0.854133	0.746200
5	Extreme Gradient Boosting	211.245867	4.107498e+05	637.078433	0.849367	0.780567
6	K Neighbors Regressor	347.010167	6.644505e+05	812.615700	0.753200	0.126967
7	Linear Regression	418.807700	7.449175e+05	862.479100	0.721900	0.032533
8	Bayesian Ridge	417.863867	7.451681e+05	862.610867	0.721867	0.017667
9	Ridge Regression	416.316667	7.460611e+05	863.093300	0.721600	0.024133
10	Lasso Regression	414.664600	7.468230e+05	863.591133	0.721200	0.028333
11	Least Angle Regression	414.671567	7.468090e+05	863.582867	0.721200	0.011600
12	Lasso Least Angle Regression	414.671567	7.468090e+05	863.582867	0.721200	0.096300
13	Decision Tree Regressor	281.563167	7.666359e+05	874.577667	0.714400	0.031600
14	Huber Regressor	366.026233	8.867765e+05	939.992433	0.671133	0.587967
15	AdaBoost Regressor	630.970500	9.018329e+05	949.130033	0.658400	0.167300
16	Orthogonal Matching Pursuit	390.619300	9.338198e+05	964.849833	0.653000	0.042933
17	Passive Aggressive Regressor	475.955233	9.786981e+05	987.068733	0.637333	0.040333
18	Elastic Net	469.752767	1.237737e+06	1109.245500	0.542800	0.007633
19	Dummy Regressor	851.671500	2.698218e+06	1639.846800	-0.001033	0.000300

Results table showing the performance metrics of different regression models in test dataset

In [ ]:
```
test_results = pd.DataFrame(
    {
        "Model": list(models.keys()),
        "MAE": test_mae,
        "MSE": test_mse,
        "RMSE": test_rmse,
        "R2": test_r2,
        "TT (Sec)": test_tt,
    }
)

test_results.sort_values("R2", axis=0, ascending=False).reset_index(drop=True)
```

Out[ ]:

	Model	MAE	MSE	RMSE	R2	TT (Sec)
0	CatBoost Regressor	206.8450	3.519263e+05	593.2338	0.8738	0.0038
1	Random Forest Regressor	226.8002	3.899239e+05	624.4389	0.8602	0.0294
2	Extreme Gradient Boosting	219.3328	4.028887e+05	634.7352	0.8556	0.0060
3	Gradient Boosting Regressor	246.8139	4.269682e+05	653.4281	0.8469	0.0023
4	Extra Trees Regressor	223.1459	4.290092e+05	654.9879	0.8462	0.0311
5	Light Gradient Boosting Machine	231.0250	4.462563e+05	668.0242	0.8400	0.0039
6	Decision Tree Regressor	274.8801	5.511628e+05	742.4034	0.8024	0.0020
7	K Neighbors Regressor	360.9694	7.398412e+05	860.1402	0.7348	0.0159
8	Ridge Regression	447.5752	8.176918e+05	904.2631	0.7069	0.0019
9	Bayesian Ridge	448.9835	8.183888e+05	904.6484	0.7066	0.0018
10	Linear Regression	449.7303	8.190990e+05	905.0409	0.7064	0.0020
11	Least Angle Regression	447.3380	8.209981e+05	906.0895	0.7057	0.0019
12	Lasso Regression	447.3344	8.210098e+05	906.0959	0.7057	0.0018
13	Lasso Least Angle Regression	447.3380	8.209981e+05	906.0895	0.7057	0.0031
14	Huber Regressor	395.6735	9.764732e+05	988.1666	0.6500	0.0035
15	AdaBoost Regressor	725.5011	1.021388e+06	1010.6374	0.6339	0.0132
16	Orthogonal Matching Pursuit	427.7810	1.066703e+06	1032.8132	0.6176	0.0017
17	Passive Aggressive Regressor	489.4734	1.110461e+06	1053.7839	0.6019	0.0017
18	Elastic Net	484.7143	1.284175e+06	1133.2146	0.5397	0.0037
19	Dummy Regressor	859.7680	2.790455e+06	1670.4655	-0.0003	0.0001

Insight 💡

- The CatBoost Regressor has the highest R2 value, indicating that it explains the variance in the target variable the best among all the models.
- The CatBoost Regressor has the lowest MAE, MSE, and RMSE among all the models, which indicates that it is the best-performing model in terms of accuracy.
- The Random Forest Regressor, Extreme Gradient Boosting, and Extra Trees Regressor models also perform well and have R2 values above 0.84.
- The Linear Regression, Ridge Regression, and Bayesian Ridge models have similar performance, with R2 values around 0.70 and relatively high MAE, MSE, and RMSE values.
- The Dummy Regressor has a negative R2 value, indicating that it is worse than the mean predictor in terms of accuracy.
- The training time (TT) is generally low for most of the models, with the highest value being 0.0311 seconds for the Extra Trees Regressor.

Overall, the CatBoost Regressor is the best model for this dataset, as it has the lowest error metrics and the highest R2 value.

Training CatBoostRegressor model

Training the model to get more insights

In [ ]:
```
regressor = CatBoostRegressor(verbose=0, allow_writing_files=False)
```

In [ ]:
```
regressor.fit(X_train, y_train)
```

Out[ ]:
```
<catboost.core.CatBoostRegressor at 0x7f79463e2e00>
```

Feature importance

Using `regressor.feature_importances_` we will get which feature is important or not

In [ ]:
```
transform_df = pd.DataFrame(
    [regressor.feature_importances_],
    columns=[
        "Shape_CUSHION",
        "Shape_EMERALD",
        "Shape_HEART",
        "Shape_MARQUISE",
        "Shape_OVAL",
        "Shape_PEAR",
        "Shape_PRINCESS",
        "Shape_ROUND",
        "Clarity",
        "Colour",
        "Cut",
        "Polish",
        "Symmetry",
        "Fluorescence",
        "Weight",
        "Length",
        "Width",
        "Depth",
    ],
    index=["feature_importances"],
)
transform_df = transform_df.T
transform_df.sort_values(by=["feature_importances"], ascending=False, inplace=True)
transform_df
```

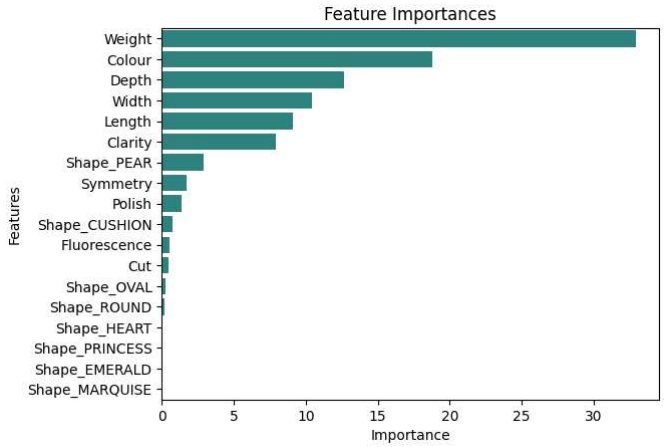
Out[ ]:

	feature_importances
Weight	32.934920
Colour	18.784300
Depth	12.676299
Width	10.458805
Length	9.102871
Clarity	7.906486
Shape_PEAR	2.912063
Symmetry	1.700261
Polish	1.345479
Shape_CUSHION	0.719666
Fluorescence	0.537803
Cut	0.432661
Shape_OVAL	0.253103
Shape_ROUND	0.163249
Shape_HEART	0.051964
Shape_PRINCESS	0.009143
Shape_EMERALD	0.007185
Shape_MARQUISE	0.003742

In [ ]:
```
sns.barplot(
    x="feature_importances", y=transform_df.index, data=transform_df, color="#21918c"
)

# set the title and axis labels
plt.title("Feature Importances")
plt.xlabel("Importance")
plt.ylabel("Features")

# show the plot
plt.show()
```



Insight 💡

- The "Weight" feature has the highest importance at 32.93%, followed by "Colour" at 18.78% and "Depth" at 12.67%.
- "Width" and "Length" have importance levels of 10.45% and 9.10%, respectively.
- "Clarity" and "Shape_PEAR" have importance levels of 7.91% and 2.91%, respectively.
- "Symmetry", "Polish", "Shape_CUSHION", and "Fluorescence" have relatively lower importance levels, ranging from 1.70% to 0.53%.
- The remaining features, including "Cut", "Shape_OVAL", "Shape_ROUND", "Shape_HEART", "Shape_PRINCESS", "Shape_EMERALD", and "Shape_MARQUISE" have very low importance levels, all below 1%.

Conclusion 🎉

In this notebook, we performed several regression models for our project and selecting the best one. We started by importing necessary libraries, loading the dataset, and applied several regression models and selecting best model using k-folds.

By observation, we can conclude that CatBoost Regressor is the best model for this dataset, as it has the lowest error metrics and the highest R2 value.

Finally, we also got insights about feature importance. Overall, these steps helped us to create base model which makes it ready for further pipeline.